МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра ИИТ

Отчет по лабораторной работе №5

Специальность ПО-12

Выполнил:
Студент 3 курса
Группы ПО-12
Мельник Р.С.
Проверила:
Кулик А. Д..

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Задание:
1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Вариант 23: База данных Бухгалтерия

Код программы:

```
import datetime

from typing import List

#http://127.0.0.1:8000/docs

#uvicorn task:app --reload

from fastapi import FastAPI, Depends, HTTPException

from pydantic import BaseModel

from sqlalchemy import create_engine, Column, Integer, String, Float, ForeignKey, DateTime

from sqlalchemy.orm import sessionmaker, relationship, Session, declarative_base



SQLALCHEMY_DATABASE_URL = "sqlite:///./accounting.db"

engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class Department(Base):
    __tablename__ = "departments"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, unique=True)
    employees = relationship("Employee", back_populates="department")

class Employee(Base):
    __tablename__ = "employees"
    id = Column(Integer, primary_key=True, index=True)
    full_name = Column(String)
    position = Column(String)
    salary = Column(Float)
    dept_id = Column(Integer, ForeignKey("departments.id"))

    department = relationship("Department", back_populates="employees")

class Counterparty(Base):
    __tablename__ = "counterparties"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    inn = Column(String, unique=True)

class Category(Base):
```

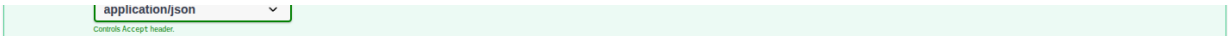
```
    __tablename__ = "categories"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)

class Transaction(Base):
    __tablename__ = "transactions"
    id = Column(Integer, primary_key=True, index=True)
    amount = Column(Float)
    description = Column(String)
    created_at = Column(DateTime, default=datetime.datetime.now)

    emp_id = Column(Integer, ForeignKey("employees.id"))
    counterparty_id = Column(Integer, ForeignKey("counterparties.id"))
    category_id = Column(Integer, ForeignKey("categories.id"))

Base.metadata.create_all(bind=engine)




class EmployeeCreate(BaseModel):
    full_name: str
    position: str
    salary: float
    dept_id: int

class EmployeeOut(EmployeeCreate):
```

```
    id: int


    class Config:
        orm_mode = True





app = FastAPI(title="Система Бухгалтерии (Accounting API)")


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


@app.get("/employees/", response_model=List[EmployeeOut],
summary="Получить список всех сотрудников")
def read_employees(db: Session = Depends(get_db)):
    return db.query(Employee).all()


@app.post("/employees/", response_model=EmployeeOut, summary="Добавить
нового сотрудника")
def create_employee(employee: EmployeeCreate, db: Session = Depends(get_db)):
    db_employee = Employee(**employee.dict())
    db.add(db_employee)
```

```
    db.commit()

    db.refresh(db_employee)

    return db_employee


@app.put("/employees/{emp_id}", summary="Изменить зарплату сотрудника")

def update_employee(emp_id: int, new_salary: float, db: Session =
Depends(get_db)):

    db_emp = db.query(Employee).filter(Employee.id == emp_id).first()

    if not db_emp:

        raise HTTPException(status_code=404, detail="Сотрудник не найден")


    db_emp.salary = new_salary

    db.commit()

    return {"message": f"Зарплата сотрудника ID={emp_id} успешно обновлена
на {new_salary}"}


@app.delete("/employees/{emp_id}", summary="Уволить (удалить)
сотрудника")

def delete_employee(emp_id: int, db: Session = Depends(get_db)):

    db_emp = db.query(Employee).filter(Employee.id == emp_id).first()

    if not db_emp:

        raise HTTPException(status_code=404, detail="Сотрудник не найден")


    db.delete(db_emp)

    db.commit()

    return {"message": "Сотрудник удален из базы"}


@app.post("/setup/", summary="Заполнить справочники (отделы, категории)")
```

```
def setup_db(db: Session = Depends(get_db)):
    if db.query(Department).first():
        return {"message": "База уже инициализирована"}

    dept1 = Department(name="Бухгалтерия")
    dept2 = Department(name="IT Отдел")
    cat1 = Category(name="Выплата зарплаты")
    cp1 = Counterparty(name="Банк ВТБ", inn="123456789")

    db.add_all([dept1, dept2, cat1, cp1])
    db.commit()
    return {"message": "Базовые данные (отделы, категории, контрагенты) успешно добавлены!"}
```

Результат:



No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "full_name": "Иванов Иван",
  "position": "Бухгалтер",
  "salary": 15000,
  "dept_id": 1
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/employees/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "full_name": "Иванов Иван",
  "position": "Бухгалтер",
  "salary": 15000,
  "dept_id": 1
}'
```

Request URL

```
http://127.0.0.1:8000/employees/
```

Server response

default

GET /employees/ Получить список всех сотрудников

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/employees/' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/employees/
```

Server response

Code	Details
200	Response body

```
[
  {
    "full_name": "Иванов Иван",
    "position": "Бухгалтер",
    "salary": 15000,
    "dept_id": 1,
    "id": 1
  }
]
```

Download

Response headers

```
content-length: 107
content-type: application/json
date: Tue,31 Mar 2026 11:01:56 GMT
server: uvicorn
```

Responses

Code	Description	Links

PUT /employees/{emp_id} Изменить зарплату сотрудника

Parameters

Cancel

Name	Description
emp_id * required integer (path)	1
new_salary * required number (query)	20000

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/employees/1?new_salary=20000' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/employees/1?new_salary=20000
```

Server response

Code	Details
200	Response body

```
{
  "message": "Зарплата сотрудника ID=1 успешно обновлена на 20000.0"
}
```

Download

Response headers

```
content-length: 103
content-type: application/json
```

```
}
```

DELETE `/employees/{emp_id}` Уволить (удалить) сотрудника

Parameters

Cancel

Name	Description
emp_id * required integer (path)	1

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/employees/1' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/employees/1
```

Server response

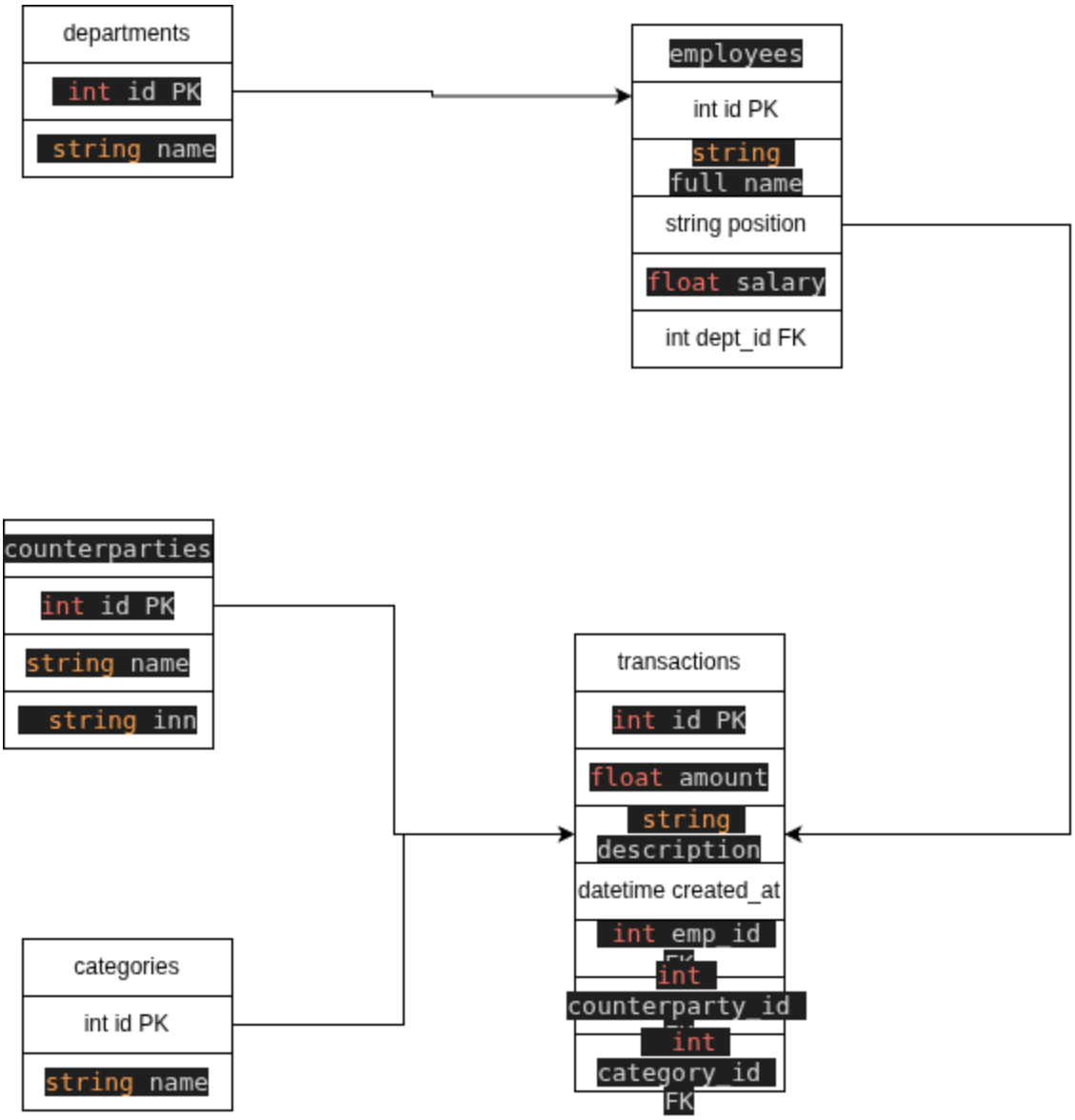
Code	Details
200	Response body

```
{
  "message": "Сотрудник удален из базы"
}
```

Download

Response headers

```
content-length: 59
content-type: application/json
date: Tue,31 Mar 2026 11:02:46 GMT
server: uvicorn
```



Вывод: приобрели практические навыки разработки API и баз данных